

컴퓨터비전과 기계학습

10주차 1차시

7장-매칭(2)

학습목차

여백

여백

7-2-1 kd 트리 (k-dimensional Tree)

■ 이진검색 트리 (BST Binary Search Tree)

- 루트를 기준으로 왼쪽 부분 트리는 루트보다 작은 값, 오른쪽은 큰 값을 갖는 이진 트리
(이 성질이 부분 트리에 재귀적으로 반복 적용)

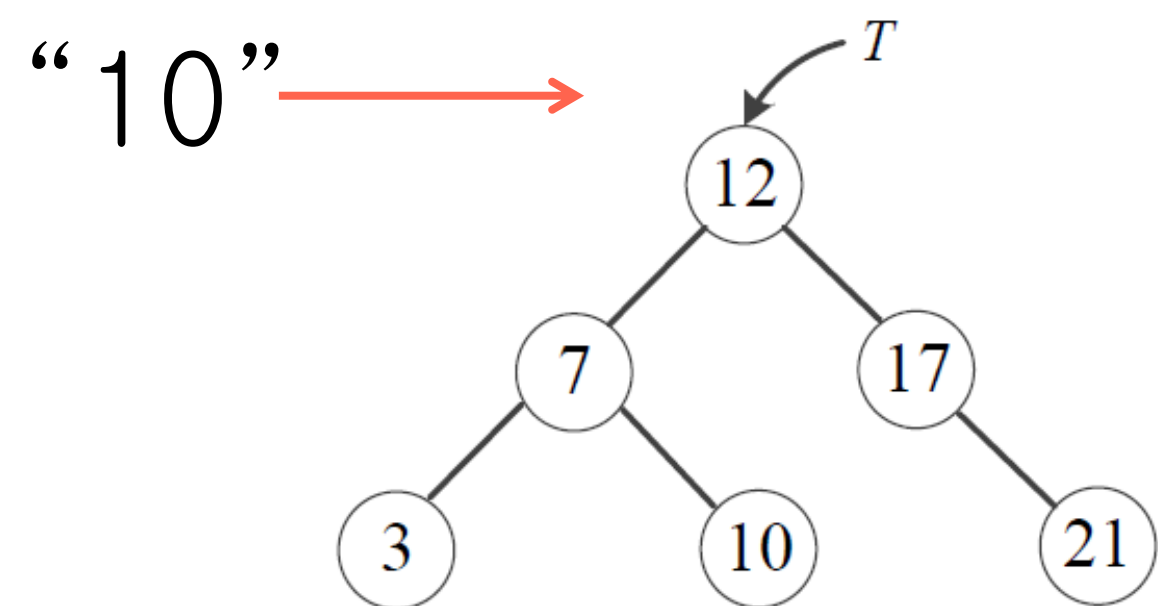


그림 7-5 이진검색 트리

균형이 잡힌 경우
탐색시간 $O(\log n)$

알고리즘 7-3 이진검색 트리의 검색

입력 : 이진검색 트리 T , 검색 키 v

출력 : 검색 결과

```
1  t=search(T, v);
2  if(t=Nil) T 안에 v가 없음을 알린다.
3  else t를 검색 결과로 취한다.
4  function search(t,v) {
5      if(t=Nil or t.key=v) return t;
6      else {
7          if(v<t.key) return search(t.leftchild, v);
8          else return search(t.rightchild, v);
9      }
10 }
```


7-2-1 kd 트리

■ 매칭에 ‘이진검색트리(BST)’를 적용할 수 있나?

■ 매칭 문제

- 검색 키(특징 벡터)가 여러 개의 실수로 구성된 벡터임
- 동일한 값을 갖는 노드를 찾는 것이 아니라, 최근접 이웃을 찾음
→ BST를 그대로 적용 불가능

■ kd 트리

- 두 가지 다른 점을 수용할 수 있게 BST를 확장한 기법 [Bently75]

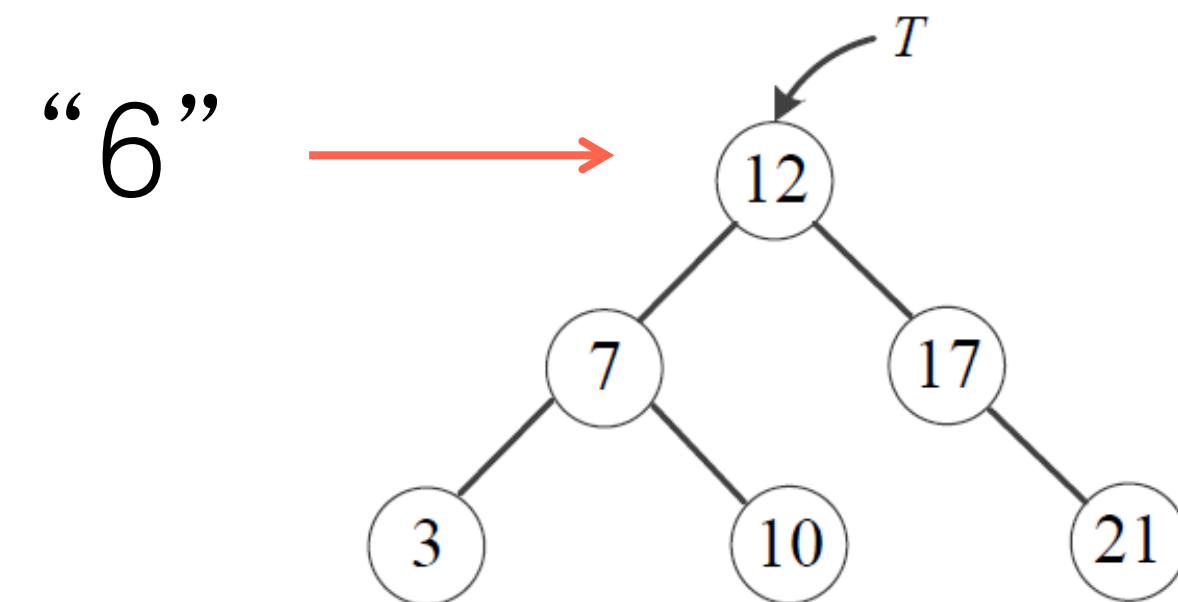


그림 7-5 이진검색 트리

7-2-1 kd 트리

■ 표기

- n 개의 특징 벡터 X 를 가지고 kd 트리 구축 $\rightarrow X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$
- 여기서 하나의 특징벡터 $\mathbf{x}_i$ 는 $\mathbf{x}_i = \{x_1, x_2, \dots, x_d\}$: d 차원 벡터

7-2-1 kd 트리

■ kd 트리의 원리

- 루트 노드는 X 를 두 개의 부분 집합 X_{left} 와 X_{right} 나눔
- 이때 분할 기준을 어떻게 선택하나?
 - d 개의 차원(축) 중에 어느 것을 쓸 것인가?
 - 분할 효과를 극대화하려면 각 차원의 분산을 계산한 후, 최대 분산을 갖는 축 k 를 선택
 - 축을 선택했다면 n 개의 샘플 중 어느 것을 기준으로 X 를 분할할 것인가?
 - X_{left} 와 X_{right} 의 크기를 갖게 하여 균형 잡힌 트리를 만들어야 함.
 - X 를 차원 k 로 정렬하고, 그 결과의 중앙 값을 분할 기준으로 삼는다.
- X 를 X_{left} 와 X_{right} 로 분할한 후, 각각에 같은 과정을 재귀적으로 반복하면 kd트리 완성

7-2-1 kd 트리

알고리즘 7-4 kd 트리 만들기

입력 : 특징 벡터 집합 $X = \{x_i, i=1, 2, \dots, n\}$

출력 : kd 트리 T

```
1  T=make_kdtree(X);
2  function make_kdtree(X) {
3      if(X=∅) return Nil;
4      else if(|X|=1) { // 단말 노드
5          트리 노드 node를 생성한다.
6          node.vector =  $x_m$ ; //  $x_m$ 은 X에 있는 벡터
7          node.leftchild = node.rightchild = Nil;
8          return node;
9      }
10     else {
11         d개의 차원 각각에 대해 X의 분산을 구하고 최대 분산을 갖는 차원을 k라 하자.
12         X를 k차원을 기준으로 정렬하여 리스트  $X_{sorted}$ 를 만든다.
13          $X_{sorted}$ 에서 중앙값을  $x_m$ , 왼쪽 부분집합을  $X_{left}$ , 오른쪽 부분집합을  $X_{right}$ 라 하자.
14         트리 노드 node를 생성한다.
15         node.dim = k; // 어느 차원으로 분할하는지
16         node.vector =  $x_m$ ; // 어떤 특징 벡터로 분할하는지
17         node.leftchild = make_kdtree( $X_{left}$ );
18         node.rightchild = make_kdtree( $X_{right}$ );
19         return node;
20     }
21 }
```

7-2-1 kd 트리

예제 7-3 kd 트리 만들기

설명을 쉽게 하기 위해 $d=2$ 로 한정하고, $X=\{x_1=(3,1), x_2=(2,3), x_3=(6,2), x_4=(4,4), x_5=(3,6), x_6=(8,5), x_7=(7,6.5), x_8=(5,8), x_9=(6,10), x_{10}=(6,11)\}$ 이라 하자. [그림 7-6(a)]는 주어진 특징 벡터의 집합을 보여준다.

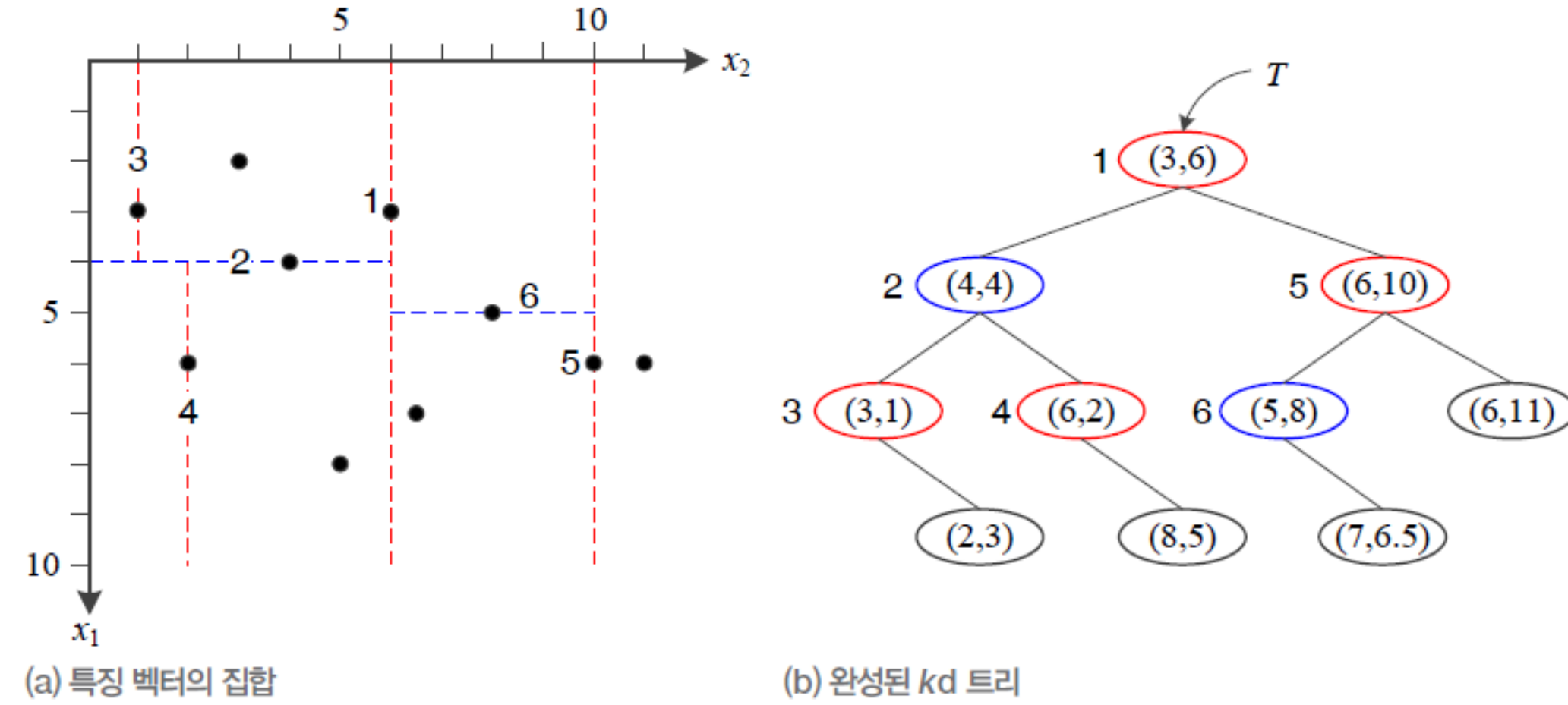


그림 7-6 kd 트리

먼저, 루트 노드로 결정할 만한 분할 기준을 찾아보자. 두 개의 차원은 각각 3, 2, 6, 4, ..., 6과 1, 3, 2, 4, ..., 11의 값을 가진다. 이들의 분산을 구해 보면 두 번째가 더 크다. 따라서 [알고리즘 7-4]의 11행에서 k 는 2가 된다. 두 번째 차원을 기준으로 X 를 정렬하면 $X_{sorted}=\{x_1, x_3, x_2, x_4, x_6, x_5, x_7, x_8, x_9, x_{10}\}$ 이 된다. 이 리스트의 중앙값 x_5 를 기준으로 좌우를 분할하면, $X_{left}=\{x_1, x_3, x_2, x_4, x_6\}$, $X_{right}=\{x_7, x_8, x_9, x_{10}\}$ 이 된다. 이제 14~16행에서 노드를 하나 할당받아 값을 채운다. 이렇게 만들어진 노드가 [그림 7-6]에서 T 가 가리키는 루트 노드이다.

이 루트 노드의 물리적인 의미를 해석해 보자. [그림 7-6(a)]에서 1 옆의 빨간색 선이 이 노드의 역할을 보여준다. 이 노드는 $k=2$ 에 해당하는 x_2 축을 기준으로 공간을 둘로 분할한다. 이때 왼쪽 영역에 있는 점들이 X_{left} 가 되고 오른쪽 영역은 X_{right} 가 된다. 이제 X_{left} 와 X_{right} 각각에 같은 과정을 재귀적으로 반복하면 [그림 7-6(b)]와 같은 kd 트리가 완성된다. 그림에서는 기준이 되는 축을 쉽게 구분할 수 있도록 각각 다른 색으로 표시하였다. x_1 축이 기준이라면 파란색, x_2 축이 기준이라면 빨간색이다.

7-2-1 kd 트리

■ kd 트리에서 최근접 이웃 탐색

- 새로운 특징 벡터 x 가 입력되면 x 의 최근접 이웃을 어떻게 찾을까? ③④
- 예) x 가 $(7, 5.5)$
 - 루트 노드는 ②축 기준 : 루트가 x_2 축을 기준으로 하므로 5.5는 분할 기준값 6과 비교하여 작으므로 왼쪽으로 분기
 - $(4,4)$ 노드(① 노드)가 x_1 축을 기준으로 하므로 7과 4를 비교하고 크므로 오른쪽으로 분기
 - 반복하면 $(8,5)$ 노드에 도착

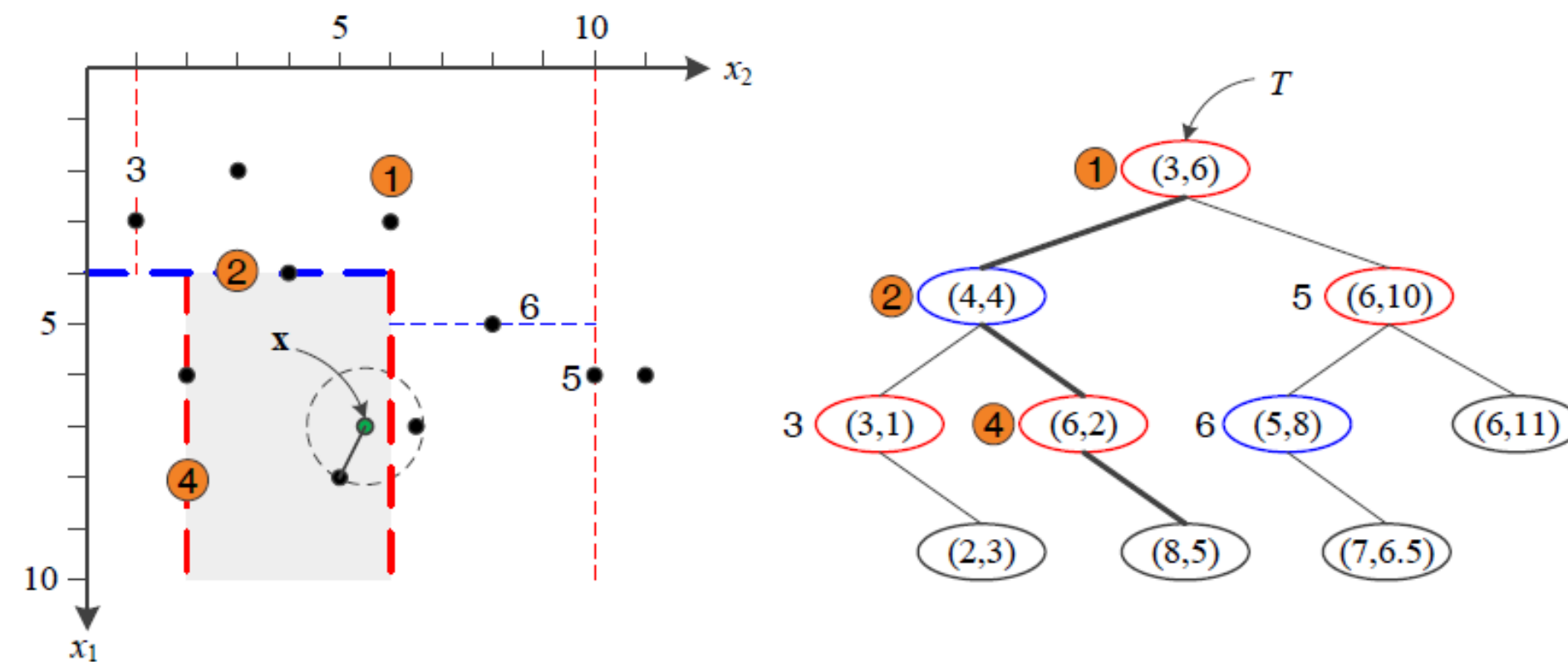


그림 7-7 kd 트리에서 검색하는 예

7-2-1 kd 트리

알고리즘 7-5 kd 트리에서 최근접 찾기

입력 : kd 트리 T , 탐색할 특징 벡터 x

출력 : 최근접 이웃 $nearest$

```
1  stack s=∅; // 백트래킹을 위해, 지나온 노드를 저장할 스택을 생성한다.
2  current_best=∞;
3  nearest=Nil;
4  search_kdtree( $T, x$ );
5  function search_kdtree( $T, x$ ) {
6     $t=T$ ;
7    while(not is_leaf( $t$ )) { // 단말 노드를 찾는다.
8      if( $x[t.dim] < t.vector[t.dim]$ ) {s.push( $t, "right"$ );  $t=t.leftchild$ ;}
9      else {s.push( $t, "left"$ );  $t=t.rightchild$ ;}
10   }
11    $d=dist(x, t.vector)$ ;
12   if( $d < current\_best$ ) { $current\_best=d$ ;  $nearest=t$ ;}
13   while( $s \neq \emptyset$ ) { // 거쳐온 노드 각각에 대해 최근접 가능성 여부를 확인(백트래킹)
14     ( $t_1, other\_side$ )=s.pop();
15      $d=dist(x, t_1.vector)$ ;
16     if( $d < current\_best$ ) { $current\_best=d$ ;  $nearest=t_1$ ;}
17     if( $x$ 에서  $t_1$ 의 분할 평면까지 거리  $< current\_best$ ) { // 건너편에 더 가까운 것 있을 수 있음
18        $t_1$ 의  $other\_side$  자식 노드를  $t_{other}$ 라 하자.
19       search_kdtree( $t_{other}, x$ );
20     }
21   }
22 }
```

7-2-1 kd 트리

- kd 트리를 이용한 최근접 이웃 탐색
 - 리프 노드 (8,5)를 답으로 취하면 될까?
 - 최근접일 가능성이 있지만 반드시 그렇지 않다. 분할 평면의 건너편에 더 가까운 노드가 있을 수 있음
 - 탐색 차원이 높아질수록 탐색량 기하 급수적 증가
- 10차원 이상 즉 $d=10$ 을 넘으면 순진한 알고리즘과 비슷한 낮은 속도
 - 어떻게 시간 효율을 회복할 수 있을까?
 - GPU 를 활용한 병렬처리의 필요성

7-2-1 kd 트리

■ 근사 최근접 이웃 탐색

■ 최적 칸 우선 탐색

- **스택** (stack) 대신 우선순위 **큐** (Queue)인 힙(heap)을 사용

- 거리를 우선순위로 사용하는데, 백트래킹할 때 가까운 것부터 조사하도록 해줌

- 예) 그림 7-7에서, ① → ④ → ② 순으로 처리

- 물리적 거리 < 최근접 이웃에 놓여있을 가능성이 큰

- 미리 설정해 놓은 값 *try_allowed* (임계값의 성격)에 따라 조사 횟수를 제한함

- 근사 최근접에서 멈춤 (최적칸 우선을 적용했기 때문에 최근접 찾을 확률 높음)

- 대신 시간 효율 얻음

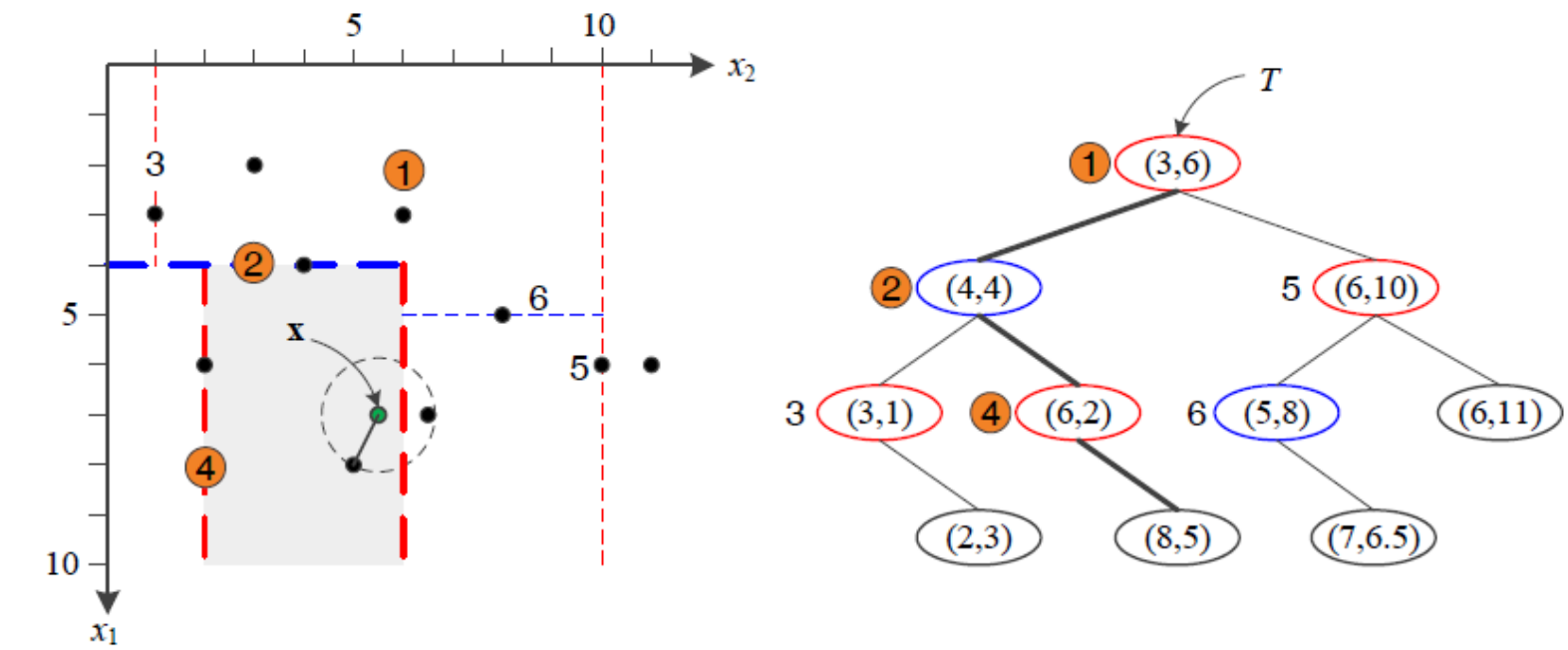


그림 7-7 kd 트리에서 검색하는 예

7-2-1 kd 트리

- 성능: SIFT의 실험 예 [Lowe2004]
 - $try_allowed=200$ 으로 했을 때, $n=100,000$, $d=128$ 인 상황에서 최근접 95%, 근사 최근접 5%
 - 속도는 100배 빨라짐

7-2-1 kd 트리

알고리즘 7-6 kd 트리에서 근사 최근접 이웃 찾기

입력 : kd 트리 T , 탐색할 특징 벡터 $\mathbf{x}$, 허용된 최대 조사 횟수 $try_allowed$

출력 : 근사 최근접 이웃 $nearest$

```
1  heap s=∅; // 백트래킹을 위해, 지나온 노드를 저장할 힙을 생성한다.
2  current_best=∞;
3  nearest=Nil;
4  try=0; // 비교 횟수를 센다.
5  search_kdtree( $T, \mathbf{x}$ )
6  function search_kdtree( $T, \mathbf{x}$ ) {
7       $t=T$ ;
8      while(not is_leaf( $t$ )) { // 단말 노드를 찾는다.
9          if( $\mathbf{x}[t.dim] < t.vector[t.dim]$ ) {s.push( $t, "right"$ );  $t=t.leftchild$ ;}
10         else {s.push( $t, "left"$ );  $t=t.rightchild$ ;}
11     }
12      $d=dist(\mathbf{x}, t.vector)$ ;
13     if( $d < current\_best$ ) { $current\_best=d$ ;  $nearest=t$ ;}
14     try++;
15     if( $try > try\_allowed$ ) 알고리즘을 끝낸다.
16     while( $s \neq \emptyset$ ) { // 거쳐온 노드 각각에 대해 최근접 가능성 여부를 확인(백트래킹)
17         ( $t_1, other\_side$ )=s.pop();
18          $d=dist(\mathbf{x}, t_1.vector)$ ;
19         if( $d < current\_best$ ) { $current\_best=d$ ;  $nearest=t_1$ ;}
20         if( $\mathbf{x}$ 에서  $t_1$ 의 분할 평면까지 거리  $< current\_best$ ) { // 건너편에 더 가까운 것 있을 수 있음
21              $t_1$ 의  $other\_side$  노드를  $t_{other}$ 라 하자.
22             search_kdtree( $t_{other}, \mathbf{x}$ );
23         }
24     }
25 }
```

← 허용된 만큼만 백트래킹

7-2-2 해싱

- 해싱의 원리
 - 해시 함수는 키 값을 해시 테이블의 주소로 변환
 - 테이블에 골고루 배치할수록 좋은 해시 함수
 - 충돌 해결책 필요

해시 함수 $h(x)=x \bmod 13$

0	
1	27
2	
3	
4	147
5	
6	19
7	
8	8
9	
10	23
11	1311
12	

그림 7-8 해싱의 원리